DQ-Stack

Probabilistic demand forecasting and inventory decisions

A complete reference: what the system does, how the eight-stage pipeline works, the architecture and its constraints, the full design system, every screen in the interface, the measured results, and the engineering record.

Series planned	1,000
Panel rows	1,128,000
Forecast horizon	28 days, direct multi-horizon
Quantile levels	9 per day per series
Backtest origins	8 rolling, paired inference
Pipeline	8 stages, 7,946 lines of Python
Interface	8,822 lines, no build step, no framework
Run generated	2026-08-06 14:23:02

Every figure in this document is read from `artifacts/` and `ui/css/tokens.css` at build time. Nothing is transcribed.

Contents

	Part	Contents
I	What the system is	The problem, the thesis, and who uses it
II	The eight-stage pipeline	Every stage: purpose, method, and why
III	Architecture	Constraints, the read path, and in-browser SQL
IV	The design system	Identity, colour, type, space, motion
V	The interface	Every view, control and keyboard path
VI	Replenishment	Stock counts, the order list, and cycle counting
VII	Evidence and results	Accuracy, calibration, and economics
VIII	Engineering record	Defects found, budgets measured
IX	Limits	What is deliberately not built

This document describes a working system. Where a claim is measured, the measurement is quoted; where something is unproven or deliberately absent, it is stated as such rather than omitted.

Part I

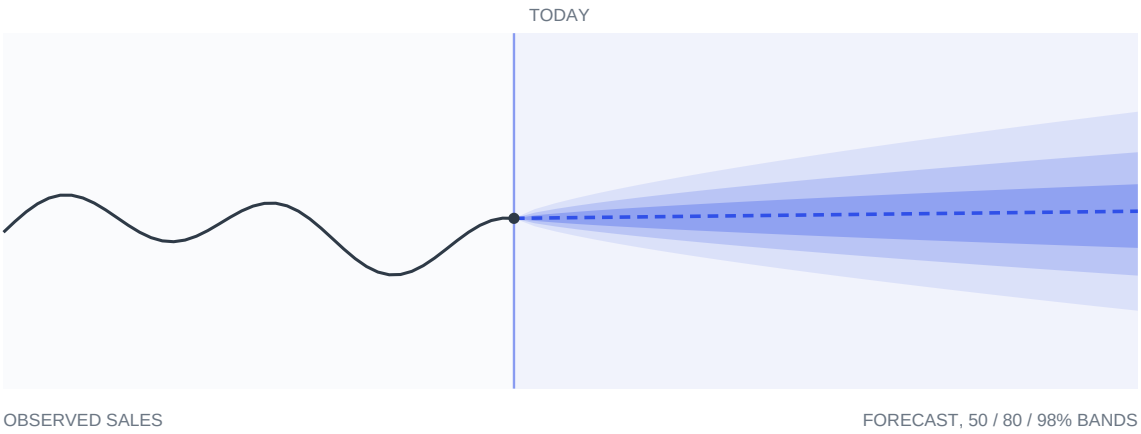
What the system is

A demand planner at a grocery or retail chain decides how much of every product to order for every store, every week. One planner may be accountable for thousands of product-and-store combinations. They cannot look at them one at a time, and the consequences of being wrong are asymmetric and specific to each product: running out of a high-margin durable costs a sale that will not come back, while over-ordering fresh milk means throwing it away.

The thesis

Most forecasting tools produce a number and leave the planner to trust it. This system argues that a forecast is only useful when four things travel with it: **how uncertain it is**, **whether the data behind it can be trusted**, **what decision follows from it in money**, and **what it costs when a human overrules it**. Every architectural choice below follows from that.

The sharpest expression is the ordering rule. Two products with an identical forecast get different orders, because the right stock level depends on what mistakes cost. The system derives a service level per product from its own price, cost, shelf life and holding cost - the critical fractile, underage over underage plus overage - instead of applying one blanket target to everything. Measured across this assortment the target ranges from **50%** to **85%**, with a mean of **0.711** and a median of 0.718. A blanket policy cannot express any of that spread.



The motif and the method are the same object. Observed sales resolve into a forecast that widens as it reaches further out, drawn as three nested bands - the middle 50%, 80% and 98% of plausible outcomes. The interface never shows a forecast without it.

Who uses it

Role	What they do here
Demand planner	Works the daily decision queue, approves or overrides forecasts, adds and retires products. The primary user.
Category manager	Reviews performance and economics by category, department and store.

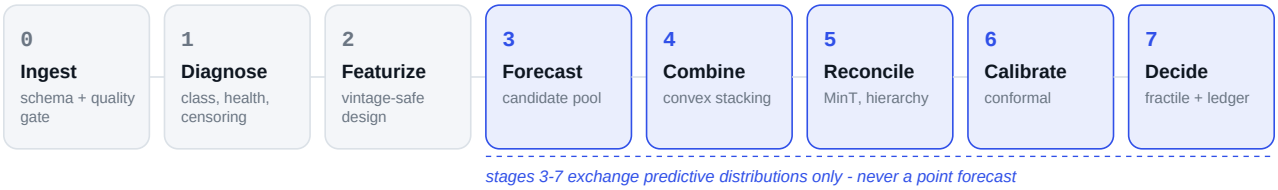
Role	What they do here
Supply-chain analyst	Interrogates accuracy, calibration and model attribution; queries the planning table directly in SQL.
Reviewer or examiner	Reads the evidence surfaces: forecast value added by stage, reliability, and the accuracy-versus-economics comparison.

How it works, in one paragraph

Two years of daily sales go in. The system first decides whether each zero in that history means *nobody wanted it* or *the shelf was empty*, because those look identical in the data and mean opposite things. It sorts every product by how it sells - steadily, in bursts, occasionally, or too new to tell - and routes each to methods suited to that behaviour. It forecasts the next 28 days as a distribution rather than a number, combines several models, makes the forecasts add up across the product and store hierarchy, corrects the intervals against their own historical coverage, and finally turns the distribution into a stock level using that product's economics. A planner then sees only the decisions that carry real money, ranked by how much.

Part II

The eight-stage pipeline



Each stage has a single responsibility and a typed interface. The contract that makes the rest possible is that **stages three through seven exchange predictive distributions only**. Nothing downstream ever receives a point forecast, which is what allows calibration, economic evaluation and the decision layer to exist at all.

Stage 0 · Ingest and validate

Declared schemas with fingerprints, a quality gate, and reconciliation of aggregate totals against source-declared totals. A feed that fails is rejected rather than silently degraded, and every run records which feeds were accepted.

Why. A pipeline that accepts a broken feed produces a forecast that looks reasonable and is wrong. Rejection has to be a first-class outcome.

Stage 1 · Diagnose

Classifies every series by the Syntetos-Boylan-Croston quadrants (ADI 1.32, CV-squared 0.49) plus cold-start and terminal states; scores data health; detects censoring both structurally (no price record means the product was not carried) and probabilistically (weekday zero-rate); and finds regime changes with PELT changepoint detection.

Why. A single global model scored on the average of these classes hides exactly the sparse-series behaviour that most needs routing around.

Stage 2 · Featurize

Dense-matrix featurisation with a strict vintage discipline: features for an origin are computed from data available at that origin only, including the Stage-1 diagnostics, which are recomputed per origin block rather than taken from full history.

Why. The most expensive class of bug in forecasting is leakage, and it flatters results silently. There is a dedicated test suite for it.

Stage 3 · Forecast

A candidate pool rather than a single model: LightGBM Tweedie plus nine native quantile heads, the Croston family (Croston, SBA, TSB, ADIDA), Holt-Winters ETS, NHITS, a covariate-only zero-shot prior, and cohort lifecycle curves. Direct multi-horizon in three horizon groups, never recursive.

Why. Recursive forecasting compounds its own error. Direct horizons cost more to train and are honest about what they predict.

Stage 4 · Combine

Convex stacking that minimises pinball loss on the blended quantiles, solved by SLSQP, with weights conditioned on Stage-1 features.

Why. The FFORMA-style proxy is linear in the weights, so its optimum is a simplex vertex - it can only ever pick one model, never average. That is why it was replaced.

Stage 5 · Reconcile

MinT with Schaefer-Strimmer shrinkage across the product and store hierarchy, plus temporal reconciliation so daily, weekly and 28-day views are arithmetically identical.

Why. 'The weekly plan disagrees with the quarterly plan' becomes structurally impossible rather than a recurring meeting.

Stage 6 · Calibrate

Split conformal and CQR adjustment, validated out of sample. The median is never shifted, and monotonicity is restored about the median rather than by sorting.

Why. An interval that claims 90% coverage and delivers 62% will be trusted for a decision it cannot support.

Stage 7 · Decide

The critical fractile from each product's own economics, a vectorised periodic-review order-up-to simulator, and a full ledger: gross margin, holding, waste, markdown and lost margin.

Why. Accuracy is a proxy. The ledger is the thing the business actually experiences, and model promotion is gated on it.

The feedback loop

Planner overrides are recorded with a reason code and become an input to later runs: a product a human corrects every week is a product the model is failing on, and that is a measurable signal rather than an anecdote. In the current build overrides are session-local drafts exported as CSV, because there is no write path to a system of record - see Part VIII.

Part III

Architecture

Constraints

	Constraint	How it is met
CON-1	Bulk numeric transfer uses Apache Arrow IPC; JSON is not used for forecast matrices	`/forecast.arrow` and `/series.arrow`. Measured on this project's own 28,000-row matrix: 1.44 MB against 3.26 MB, and 1.1 ms against 65.6 ms to parse.
CON-2	Forecasts are versioned; a prior vintage is always available	Every series carries its previous version, drawn as a ghost line and summarised as 'versus last week'.
CON-3	Forecasts are precomputed in batch; no model inference on the read path	Every endpoint serves rows the pipeline already wrote. A new product is marked *awaiting its first forecast* rather than having one invented in the browser.
CON-4	Uncertainty is never discarded between stages	Stages 3-7 exchange distributions only.
CON-5	Numeric aggregation happens in the analytical store or the client query engine, not in application code	DuckDB views over the Parquet lake; rollups expressed as SQL.

The read path

`dqstack/serve.py` is a DuckDB analytical store over the Parquet lake, exposed as a read-only HTTP API, which also serves the workspace itself. Views rather than tables: the pipeline owns the files, and copying them into a database would create a second version of the truth that goes stale the moment a run finishes.

```
python -m dqstack.serve # workspace + API on 127.0.0.1:8000 python -m dqstack.serve --selftest # exercise the store and every route python -m dqstack.serve --measure # the Arrow versus JSON comparison
```

The static half is not a convenience. DuckDB-WASM is an ES module whose worker is cross-origin, and a page opened from `file://` has a null origin, so the browser refuses both the dynamic import and the worker. The client-side query engine cannot exist without an origin, so the app has to be served.

In-browser SQL, and what measuring it showed

The specification calls client-side SQL 'the single biggest perceived-performance win available'. It was built - a 195 KB Arrow slice registered as a DuckDB-WASM table - and then measured, and the measurement only partly agrees.

Rows	Row selection: SQL	JavaScript	Ratio
1,000	10.9 ms	0.5 ms	0.04x
10,000	35.0 ms	5.2 ms	0.15x
30,000	46.2 ms	11.6 ms	0.25x
100,000	74.5 ms	79.9 ms	1.07x - crossover
300,000	93.0 ms	215.4 ms	2.32x

Group rollups never cross over at all (0.06x at 1k, 0.22x at 300k), because a hierarchy level has three to ten distinct keys and a ten-bucket hash over a flat array is about as cheap as computation gets, while every DuckDB query pays a fixed cost. So the router chooses by the measurement rather than the claim: SQL for row selection above 100,000 rows, JavaScript for rollups and the small case, and DuckDB unconditionally for the SQL console, where the alternative is not a slower answer but no answer. Both backends are held to identical results by a self-test that runs the real workload through each.

Repository map

Path	Responsibility
dqstack/config.py	Every threshold and ablation switch
dqstack/ingest.py, schemas.py	Stage 0: schemas, fingerprints, quality gate
dqstack/diagnose.py	Stage 1: class, health, censoring, regimes
dqstack/features.py	Stage 2: vintage-safe featurisation
dqstack/models/	Stage 3: the candidate pool and the distribution contract
dqstack/combine.py	Stage 4: convex stacking
dqstack/reconcile.py	Stage 5: MinT, hierarchical and temporal
dqstack/calibrate.py	Stage 6: split conformal / CQR
dqstack/decide.py	Stage 7: fractile, simulator, ledger
dqstack/evaluate.py, backtest.py	WRMSSE, WSPL, coverage, FVA, paired inference
dqstack/serve.py	The read path: store, Arrow, HTTP, static
ui/js/plain.js	Every word a non-specialist reads
ui/js/review.js	Today: the decision queue
ui/js/catalog.js	Products: browse, detail, add and remove
ui/js/grid.js, chart.js	The canvas grid and fan chart
ui/js/duck.js	DuckDB-WASM and the two-backend resolver
ui/css/tokens.css	Design tokens - read by the canvases at paint time

Part IV

The design system

Identity: the fan

The mark is an origin rule with two nested wedges opening to the right. It was chosen because it is **already the product's own geometry** rather than a metaphor applied to it: a vertical origin and a widening cone is what the canvas chart, the canvas grid, the review sparkline and the product chart all draw. Shelf strata, the underage-overage balance and the weekly demand rhythm were considered and rejected.



- **Where it appears.** The sidebar lockup, the browser tab, the two empty states that mean 'nothing here yet', and a single cone-shaped wash on the scrolling surfaces.
- **Where it must not.** The completed-work empty state keeps a tick, because a fan there would say the opposite of what happened. Nothing else uses it - a signature that appears everywhere is wallpaper.
- **What was removed.** A film-grain texture that called itself 'the only decorative element in the interface', said nothing about the domain, and at 0.020 opacity was invisible on the panels this is actually used on.

Colour

Every hue carries exactly one meaning. The most important change in the system was splitting two meanings that had been sharing one token: `--series-forecast` meant both 'this is a predicted quantity', which the canvas renderers read, and 'this is clickable'. A hue that means two unrelated things cannot be learned, so `--ui-accent` now carries the interactive role across 38 surfaces while the data role stays put.

Forecast / demand `#4C6FFF / #3050E8`	predicted quantities, fan bands
Interactive accent `#7A5CF0 / #5B3FD4`	links, focus, selection, primary action
Observed `#C7D2DE / #2E3A47`	sales that actually happened
Prior version `#6B7684 / #98A3AF`	last cycle's forecast, as a ghost
Override `#E09B15 / #B87400`	a human changed this
Risk `#E0523C / #C0392B`	money at risk, likely to run out
Excess / good `#14A99A / #0B7D71`	healthy, or too much stock
Attention `#C99A2E / #96690B`	check this

Dark value left, light value right. Both themes are real palettes, not inversions of one another. **Forecast and interactive accent deliberately share a value today** - the split was made by adding a token with an identical value, so no pixel moved and nothing the canvas renderers resolve was touched. What it buys is the ability to move either role later without dragging the other with it.

Surfaces and contrast

Base `#0A0C11 / #F7F9FB`	the void behind everything
Surface `#10141B / #FFFFFF`	chrome, cards, panels
Raised `#171D26 / #F1F4F7`	hovers, chips, table heads
Inset `#06080C / #EDF1F5`	inputs, wells, chart grounds
Text primary `#E9EFF6 / #0E1620`	16.9:1 / 17.2:1
Text secondary `#95A3B4 / #54606D`	7.2:1 / 6.4:1
Text tertiary `#69747F / #78838F`	3.9:1 - never body text

Thirty-four contrast checks are computed across both themes at build time and all pass WCAG 2.2 AA. One finding came out of that audit: chart axis labels had been using the tertiary text colour, which measures 3.87:1 dark and 3.86:1 light on the surfaces charts actually sit on - below the 4.5:1 threshold, on the numbers that give every chart its scale. They now use the secondary colour at 7.80:1 and 5.66:1. The token itself was not revalued, because it is read by name in dozens of non-chart places.

Typography

Family	Role	Why
Instrument Sans	All interface text	Neutral, dense, legible at 10-13px.
JetBrains Mono	Every numeral in the product	A monospace face is tabular by construction, so column alignment is guaranteed rather than dependent on a font-feature setting surviving every code path.
Instrument Serif	Section headings on the three reference views	Marks the mode change from scanning an instrument to reading a document. Deliberately absent from Today, Products and the spreadsheet - the split only means something if it is exclusive.

The figure scale

This is a numbers application whose seven hero figures had used six different sizes and five tracking values, so no two panels agreed on how big 'the answer' is. Three steps now, each with the tracking that size actually wants.

Token	Size	Tracking	Weight	Used for
--fig-xl	40px	-0.03em	600	Money at risk; the exception count
--fig-lg	28px	-0.03em	600	The figure a decision acts on
--fig-md	20px	-0.02em	500	Supporting figures on a card

Words are not set in the numeral face. 'Good' or 'Fair' at 20px monospace reads as a status code rather than a judgement, and a word has no column to align to - so the figure class was split into a numeral variant and a word variant.

Space, radius, elevation, motion

Scale	Values	Rule
Spacing	2, 4, 6, 8, 12, 16, 24, 32, 48	Nine steps, nothing else. A half-step nobody named is how a spacing scale dies.
Radius	4px, 6px, 10px, 14px, pill, 0 in grid cells	Rounding blurs alignment, so grid cells have none.
Page frame	gutter 32px, rail 288px	One gutter across every view. The content edge used to move on every navigation.
Elevation	ring + shadow, three levels	On a dark surface a shadow is nearly invisible; the lit top edge is what reads as depth.
Motion	120ms feedback, 180ms panel	Confined to state changes that need confirming. Nothing animates on load or while data is being read; zeroed under prefers-reduced-motion.

The token safety contract

The canvas renderers resolve CSS custom properties **by name at paint time**. Twenty-two tokens are load-bearing: the eighteen colours cached in `util.js`, plus the two font families and the two row metrics. Renaming or removing one is not a CSS edit; it is a silent rendering failure in an unrelated file. Additions are always safe, which is why every change in this design system was made by adding a token rather than repurposing one.

Part V

The interface

Four destinations in a sidebar of nine collapsible blocks. Everything is reachable in one click; no feature is more than two.

Group	Destination	What it is for
Your work	Today	The decision queue. The landing view.
	Order list	What to buy: target minus what is on the shelf.
	Products	Browse the range, inspect one product, change the range.
Understand it	How it's doing	Accuracy, calibration and economics.
	Ordering decisions	Why two products with the same forecast get different orders.
	How it works	The eight stages, with this run's own numbers.
	Policy cost	What each of nine ways of running the system earned.
Tools	Spreadsheet	The bulk-editing instrument: canvas grid, keyboard-driven.
	Glossary, SQL console, shortcuts	Plain-English definitions; ad-hoc queries; the key map.

The sidebar

The rail holds the planner's own work rather than the application's features, which is the distinction an external review drew and the reason it grew rather than shrank. Every block carries a live number: a rail where nothing changes between visits is a list of links, and a rail where six figures moved overnight is the first thing read each morning.

It outgrew one screen once it carried replenishment as well as review, so every block is a native disclosure element whose open set is remembered per browser - a buyer lives in the order block, a demand planner lives in the queue, and neither should scroll past the other's. **A collapsed block still reports its figure in its heading**, because the whole argument for this rail is that a closed block is not a silent one.

Block	What it reports	Where it goes
Cycle	Vintage of the forecast, position in its horizon	-
Replenishment	Today's order cost over the counted share it describes	Order list
Stock take	What is due a count, by value class	Order list, filtered
Stock cover	Four buckets: runs out first, needs topping up, covered, overstocked	Order list, filtered
Your queue	The day's decisions grouped by cause, with exposure	Today, filtered
Range mix	ABC against XYZ, shaded by share of stock value	Products, filtered
Watching	Pinned products and how far they moved	Products
Saved views	Scope, filter and tab under a chosen name	Today, restored
Stores	All stores ranked by units to stock, yours pinned	Sets scope
Session	Decisions made, exposure covered, and an undo	-

Today - the decision queue

The front door shows a day's work, not the backlog. Of the flagged series, the highest-value are cut to a fixed cohort chosen once at load, so clearing the list actually finishes it - an earlier version re-sliced the queue on every render, so the progress readout went '1 of 25', then '2 of 26', and the finish line moved away as you worked.

Each item is one card asking one question, with one obvious answer. The card is built in four rungs, in the order a decision is actually made - a planner clearing twenty-four of these reads the same layout twenty-four times, so what sits where does more work here than anywhere else in the application:

Rung	Question it answers	Content
0	Is this worth my attention?	Position in the day's list, money at risk, how long the product has been selling.
1	What am I deciding?	Product and store; next week's forecast with its range; the stock-up-to level with the cost curve beside it.
2	Why is it in front of me?	The headline reason in one sentence, from a fixed priority order, and the chart behind it: 35 days of sales and the 28-day fan, bled to the card edges.
3	Do it	Approve / Change / Skip, plus the drawer and the product page.

Exactly one filled button is on screen per card. That hierarchy is what makes a card answerable in one click without reading all of it. Exposure was a hero figure and did not deserve to be: it is the reason the card sits in this position, and the position now states that directly.

Six figures came off the face of the card and into the drawer behind **W**, ordered so the two that qualify the decision come first and the four that explain the cost curve follow. A planner told that running out costs 2.2x more than overstocking, who wants to know why, finds the margin, the shelf life, the value of the line and how often it sells - which is where the answer actually is. The drawer also carries how the last 28 days went, where the number came from, and what to do.

Confidence and data-quality wording is calibrated to the terciles of this assortment rather than to guessed cutoffs - hardcoded thresholds put every visible card in one bucket, which trains a planner to ignore the field.

The cost curve

The ordering layer's argument is that two products with the same forecast deserve different stock levels, because the two ways of being wrong do not cost the same and how unequal they are is a property of the product. Said in a paragraph, nobody reads it. Drawn as a shape with the recommendation at the bottom of it, it is learnable in a week without meeting a technical term.

The curve's minimum **is** the recommended level, by construction rather than by fitting: a low point sitting beside the number printed above it would discredit both, silently. The demand distribution is rebuilt from two figures the bundle states exactly - the mean of the cover window, and F(order-up-to) = the target service level - which determines a two-parameter distribution. Verified at a worst offset of **0.000%** across all 997 series that have a forecast.

It differentiates, which is the point. Across the assortment the ratio of under-ordering cost to over-ordering cost runs 1.25 to 5.36, and the extremes are the thesis itself: 720-day-shelf, 44-53% margin lines are visibly lopsided, and 21-day-shelf, 17% margin food is visibly flat.

Changing a number

An override states a reason, and the reason is required. Every other fact about a decision can be rebuilt later - what the model said, what was chosen, what it was worth - but 'the end cap goes up on Tuesday' exists only in the head of whoever typed the number, and it is the one column that could teach a later run something. Six codes, plus free text: promotion, local event, supply, price, range, own knowledge.

A guardrail reports where the number sits, live as it is typed, and in odds rather than percentiles: 'about a 1 in 11 chance demand reaches this level'. Beyond the exported ladder it says so plainly. It is a guardrail and not a block - the planner is allowed to be right when the model is wrong, which is the whole reason an override exists. The expected effect comes from the same estimator the spreadsheet cell and the commit summary quote, so the three surfaces cannot disagree.

The work bar

A planner with twenty-four cards and forty minutes spends the same click on a $9 item as on a $16,000 one. Approve-all fixes that by discarding the judgement; a threshold keeps the judgement where the money is. Thresholds are percentiles of the list in front of the planner, and an offer survives only if it is genuinely the small tail: at most 45% of the items and at most 15% of the money. A tier with no small items correctly offers nothing rather than relabelling approve-all.

Batching is a window over the fixed cohort, never a re-slice of it. The cards on screen go from twenty-four to five while the headline, the money at risk, the progress readout and the position all stay exactly as they were.

Keyboard

Key	Action	Key	Action
J / K	Next / previous decision	W	Why this number
A or Enter	Approve	O	Open the product page
C	Change the number	?	Shortcut sheet
S	Skip	Cmd-K	Search
Cmd-Z	Undo the last decision	Esc	Close whatever is open

Undo reverses an *action*, not a row. Once one press can approve five items, undoing them one at a time is not an undo, so every decision carries the sequence number of the action that made it and a bulk approve is one unit in the undo stack exactly as it was one click in the interface. Committed decisions are deliberately not undoable, because the planner has already been told they were final.

The spreadsheet keeps its own nine bindings - arrows, range selection, fill-down, Excel-compatible copy and paste, undo and redo across the whole pending set. The letters above were chosen to be sayable rather than to mirror spreadsheet convention, because the review list is not a spreadsheet.

Finishing a session

A sticky bar reports what was decided, how many numbers were changed, and the expected margin effect - quoting the same estimator the spreadsheet uses, so the two surfaces cannot disagree. Two actions follow: **Export as CSV**, which is the only way decisions leave the browser, and **Commit**, which marks them done without claiming a write to a system of record that does not exist.

Products

A list beside a detail pane - the right layout for 'show me that product', and where search results land. The list searches the displayed name first, filters by category, store and needs-attention, and offers five orderings; the trailing figure on each row changes to match the sort, because sorting by money while showing unit counts looks like a broken sort.

The detail pane gives one product a full chart with axis labels, a prior-version ghost, its flags in plain words, an inline forecast edit that shares the spreadsheet's draft set, the model's own drivers translated out of feature names, and a peer comparison showing the same product in every other store as a p10-p90 range with a median

notch.

Adding and removing products

Two numbers a new product needs behave completely differently, and the distinction is the whole ordering thesis in miniature:

	Available immediately?	Why
Service level	Yes	It falls out of the price, cost and shelf life just entered. The browser reproduces the pipeline's own critical-fractile formula, with its constants exported from the config rather than retyped, and previews the result live as you type.
Stock level	No	An order-up-to level is a predictive quantile of demand over the risk period, and a new product has no predictive distribution. It reads 'Not yet - needs a forecast first'. An earlier version inherited a similar product's level, which contradicted the service level beside it.

Product	Margin	Shelf life	Target service
Fresh milk	17%	5 days	50%
Fresh, high margin	67%	5 days	78%
Durable, high margin	70%	2 years	93%
Durable, thin margin	8%	2 years	50%

The spreadsheet

A canvas-rendered grid, used above roughly ten thousand cells because DOM nodes stop being viable there. It carries an in-cell uncertainty micro-fan and a confidence gutter, so a row built on censored or stale data is marked before the planner reads the number. Because a canvas is invisible to assistive technology, a parallel ARIA table of the visible window is maintained alongside it. This is the bulk-editing instrument and is deliberately no longer the default surface.

Part VI

Replenishment

The system could forecast demand and compute a target - stock up to 156 units - and then stopped. It never asked the one question that turns a target into an order: how many are on the shelf now. Every screen was a recommendation nobody could act on.

On-hand is entered, never invented

Stock counts are typed by the planner and kept in the browser. They are not derived, estimated, or back-filled from the simulation, and there is deliberately **no 'fill these in for me' button**. A fabricated count flows straight into an order quantity and out of the building as a purchase, which makes this the one place in the system where a plausible-looking invention would cost real money.

So a product is in one of two states and the interface always says which. A product with no count shows a dash, never 'order 0'. Zero is a real answer meaning 'you have none', and conflating the two would hide every uncounted product inside a list of products that need nothing.

Quantity	Definition
Order quantity	stock up to - what is on the shelf. Null, not zero, when uncounted.
Days of cover	on hand / expected demand per day over the window the order covers.
Cover status	Below the lead time the shelf empties before a delivery can land; below the review period plus lead time it will not last the window this order covers; above the forecast horizon it is carrying more than the period needs.

The thresholds are the ordering policy's own windows, so they cannot drift from the logic that produced the target.

The order list

Rows are ordered by how soon the shelf empties rather than by name or by value, which is the only ordering that makes the list safe to abandon halfway through. Counts save as they are typed - a count is a fact about the world, not a proposal, so there is nothing to confirm about it, unlike a forecast override, which is a judgement and therefore has a reason attached. Enter moves to the next count, the way anyone types a stock take.

The header reports the order total and the counted share as two figures that are never added, because a confident total over a half-counted range is the number most likely to be believed and most likely to be wrong. Export produces a purchase order with a TOTAL line, separate from the decisions export because it is a different document for a different reader: one is a record of judgements, the other is a list of goods and a price.

Cycle counting

Nobody counts a whole shop at once. Counts carry a timestamp, and the recount interval comes from the ABC class the pipeline already assigns - so the lines that carry the money come round faster than the tail. Without an age, a figure entered three months ago and one entered this morning look identical, and the order list depends on both equally.

Class	Meaning	Counted every
A	Your biggest earners	7 days

Class	Meaning	Counted every
B	Middle of the range	28 days
C	The long tail	91 days

The intervals are the textbook ones and deliberately round. They are a starting policy, not a claim; what matters is that a count has an age at all and that value sets the frequency. A count whose age cannot be established is treated as due for recount, which is the safe reading.

Uncounted products are ranked by what a full shelf of each is worth - the only value figure that exists for something nobody has counted. A thousand blank fields is not a task anyone starts; naming the value that cannot become an order until they are counted, and listing the most valuable first, turns a chore into a decision about where to spend an hour.

Classification the pipeline already had

ABC ranks products by what they are worth; XYZ by how predictable they are. Both have been assigned to all thousand products since the first run and no planner-facing screen ever showed either. The rail presents them as a three-by-three grid rather than two lists, because the pairing is the point: an A product that is hard to predict needs a person, a C product that is hard to predict needs a rule, and neither list says that alone. Shade carries share of stock value, so the eye lands on where the money sits rather than on where the product count is highest - which is the long tail, always, and never the answer.

Part VII

Evidence and results

Rolling-origin backtest over 8 origins from 2026-05-19 to 2026-07-07, with **paired** inference - Wilcoxon signed-rank and an exact sign test on per-origin differences, because methods are evaluated on the same origins and an unpaired test throws that away.

Leaderboard, mean over origins

Method	WSPL	WRMSSE	MASE	Bias	Rank
DQ-Stack (full)	0.1233	0.6672	0.6456	-0.1726	1.38
no reconciliation	0.1234	0.6672	0.6456	-0.1726	1.62
lgbm_quantile	0.1250	0.6637	0.6482	-0.1374	3.00
lgbm_tweedie	0.1342	0.6923	0.6987	-0.2092	4.00
adida	0.1372	0.7160	0.6939	-0.2081	5.50
ets	0.1425	0.7357	0.7336	-0.1883	6.75
nhits	0.1442	0.7486	0.7090	-0.3234	7.75
tsb	0.1449	0.7655	0.7012	-0.2388	8.50
croston_sba	0.1467	0.7880	0.7078	-0.2897	9.88
lifecycle_cohort	0.1482	0.7280	0.7081	-0.1955	7.50

Ranked by WSPL - the weighted scaled pinball loss - because it scores the whole predictive distribution rather than only its centre. WRMSSE is shown alongside for comparability with the M5 literature.

Does the full system beat its best single model?

DQ-Stack (full) versus **lgbm_quantile** on **WSPL**: means 0.1233 against 0.1250, 8 wins to 0 across 8 origins. Wilcoxon p = 0.0078, sign test p = 0.0078. *DQ-Stack (full) better at 8/8 origins*.

DQ-Stack (full) versus **lgbm_quantile** on **WRMSSE**: means 0.6672 against 0.6637, 2 wins to 6 across 8 origins. Wilcoxon p = 0.1094, sign test p = 0.2891. *lgbm_quantile better at 6/8 origins*.

DQ-Stack (full) versus **no calibration** on **WSPL**: means 0.1233 against 0.1532, 8 wins to 0 across 8 origins. Wilcoxon p = 0.0078, sign test p = 0.0078. *DQ-Stack (full) better at 8/8 origins*.

Read honestly: the full system wins on distributional quality at every origin, and **loses to its own best single model on point accuracy**. Both are reported, because reporting only the favourable metric is the failure mode this project exists to argue against.

Calibration

Nominal	Empirical before	After	Gap before	Gap after
0.050	0.3971	0.3463	0.3471	0.2963
0.100	0.4487	0.3956	0.3487	0.2956
0.250	0.5176	0.4981	0.2676	0.2481

Nominal	Empirical before	After	Gap before	Gap after
0.500	0.6248	0.6248	0.1248	0.1248
0.750	0.7766	0.7696	0.0266	0.0196
0.900	0.9034	0.8928	0.0034	0.0072
0.950	0.9518	0.9411	0.0018	0.0089
0.975	0.9776	0.9687	0.0026	0.0063
0.990	0.9920	0.9876	0.0020	0.0024

The upper tail - the region the ordering rule actually reads a quantile from - is well calibrated, and conformal adjustment improves it further. The lower tail is not, and the table says so: at the 5% level empirical coverage is far above nominal. That is a property of a zero-inflated count distribution where a large mass sits at zero, and it matters little for a decision taken at the 50th to 95th percentile, but it is a real limitation rather than a rounding artefact.

Economics - the ledger

Demand	647,933 units
Sales	610,082 units
Lost sales	37,851 units
Fill rate	94.2%
Gross margin	$899,560
Holding cost	$22,069
Markdown cost	$40,517
Waste cost	$4,239
Lost margin	$45,939
Realised margin	**$832,735**
Conservation residual	0.00e+00 (asserted as a test)

Configuration	Realised margin	Fill rate	Avg inventory
DQ-Stack (full)	$832,735	94.2%	112
no calibration	$783,961	94.4%	162
no reconciliation	$834,199	94.7%	117
best single (LGBM-Q)	$828,146	93.8%	115
Tweedie only	$819,777	93.4%	124
seasonal naive	$790,325	92.0%	132
Croston/SBA only	$824,834	93.3%	117
full, blanket 95% service	$771,203	99.1%	205
no censoring correction	$819,415	93.0%	120

Accuracy is not economics. Across 9 configurations the rank correlation between forecast accuracy and realised margin is Spearman rho = 0.527 at p = 0.145 - positive, but not significant at this sample size. The blanket 95% service arm is the clearest case: it is as accurate as the full system by construction and materially less profitable, because it buys service nobody costed. Model promotion in this system is therefore gated on realised ledger margin, not on WRMSSE.

Reconciliation: report with it, order without it

Judged across all twelve aggregation levels, MinT improves the top and degrades the bottom, monotonically: total -0.45, state -0.36, category -0.31, store -0.19, down to item-store +0.44. Leaf-level WRMSSE goes from 0.77 to 1.20, worse than the seasonal-naive anchor at 0.98. Since Stage 7 sets an order-up-to level per item-location, the decision is made at the leaves and there the trade is against us. The decision path therefore reads the unreconciled distribution while the aggregate planning views keep the reconciled one. Reporting one number for both would have to be wrong for one of them.

Censoring detection

Precision	0.796
Recall	0.413
F1	0.544
Recall on full-zero days	0.593
Truly censored rows	31,108
Flagged	16,148

This is the one claim synthetic data supports that real M5 data cannot: ground truth for censoring exists only because the generator wrote it. Precision is high and overall recall is moderate by design - the detector is deliberately conservative, since inflating a genuine zero into imagined demand is the more expensive error.

Part VIII

Engineering record

A record of defects found and fixed is part of the argument, not an appendix to it: several of these were silently flattering results, and would have survived into a written claim if the system had not been instrumented to catch them.

Correctness defects

Defect	Consequence	Fix
The median index was computed as half the quantile count	The 75th percentile was used as the median in five places, because the quantile tuple is deliberately asymmetric.	A named constant, plus a test that greps the package for the halving idiom.
Calibration sorted the whole quantile vector	An adjusted lower quantile could be promoted into the median slot, moving a median of 5.0 to 7.0 with a zero offset.	Monotonicity restored about the median instead.
MinT was a no-op, then used the wrong error basis	Reconciling an already-coherent input gives back the input; then the covariance was estimated from bottom-up projections rather than node residuals.	Independent base forecasts per node, and residuals from those.
Spoilage and markdown were priced but never levied	The fractile charged for both while the ledger charged for neither - waste cost was exactly zero, and phantom spoilage was 54% of modelled overage.	Both levied; spoilage exposure redefined as cover in excess of shelf life.
Full-history diagnostics used as features	Leakage: Stage-1 outputs computed over all data leaked future information into every origin.	Per-origin-block vintages, with a dedicated leakage test suite.
Bias compared as a signed quantity	The most under-forecasting method was declared the winner.	Compared by magnitude.

Interface defects

Defect	Detail
Search did nothing	Choosing a result selected a row in the spreadsheet while a different view was on screen, and the search matched internal identifiers rather than the displayed name.
The grid collapsed to zero height, three times	Each time a child was added to a CSS grid without a matching template row, so a fully populated canvas painted into a zero-height box.
The progress denominator grew as you worked	The queue was re-sliced on every render, so finishing an item pulled in another.
Every card reported low confidence	Thresholds were guessed rather than derived from the distribution.
Chart strokes were twice as thick horizontally	A non-uniform viewBox scale; fixed with non-scaling-stroke.
The micro-fan double-encoded uncertainty, backwards	Band opacity was multiplied by confidence on top of band height, so the riskiest rows rendered the faintest.
A theme flip left elements in the old theme	Every property that failed was one the element declared a transition for.
Committing undid the review	The commit cleared the state that recorded what had been done.

Budgets, measured rather than asserted

Budget	Target	Measured
Application code, gzipped	under 250 KB	117.0 KB across 12 files
Grid paint, median	under 16 ms	4.2 ms cold, 3.1 ms warm
Grid paint, 95th percentile	under 16 ms	11.4 ms cold, 6.5 ms warm
Frames over 32 ms	zero	zero (worst single frame 15.8 ms)
Full dataset replacement	under 150 ms	72 ms
WCAG 2.2 AA contrast	no failures	34 checks across both themes, 0 failures
Horizontal overflow	none	0 across 6 views x 2 themes x 2 sidebar states
Arrow versus JSON, 28k rows	-	1.44 MB vs 3.26 MB; 1.1 ms vs 65.6 ms to parse

The frame budget initially failed at 18.6 ms median with eleven frames over 32 ms. Profiling ruled out the accessibility mirror and style resolution: the cost was the micro-fan, seventy-odd glyphs a frame of alpha-composited paths. Each rendered glyph is now cached to an offscreen canvas keyed by its data, and during a scroll an uncached glyph draws only its median line, upgrading to full fidelity 90 ms after motion stops. Nobody can read a 40 by 14 pixel glyph during a fast scroll, and fidelity was verified intact at rest.

Verification

Test suite	94 tests: leakage, simulator, contracts, diagnostics, backtest, sensitivity, and the read path
Continuous integration	Tests, a UTF-8 source check, a JavaScript parse check, and an end-to-end smoke run asserting every artifact
Run invariants	Unit conservation under 1e-6, no negative stock, MinT projection error under 1e-6 - asserted, not merely reported
Two-backend parity	The SQL and JavaScript query paths are diffed over the real workload on every load of the console

Part IX

Limits and what is deliberately not built

Stated plainly, because a system that hides its boundaries is making the same mistake as a forecast that hides its uncertainty.

Honest limitations

- **The data is synthetic.** M5-shaped and generated, not a real retailer's. Accuracy figures are therefore not comparable to the M5 literature, and the benchmark gate the specification sets is unmeetable on this data. The substitute claim is the censoring precision and recall, which real M5 data cannot provide because it has no ground truth for censoring.
- **Nothing is written anywhere.** Overrides, additions and removals are session-local drafts. The CSV export is the only way a decision leaves the browser. Wiring a real write path is the single largest gap between this and something usable in production.
- **Combination helps at only two of eight origins.** The candidate pool is dominated by one model, so stacking has little to work with. This is reported rather than buried.
- **Spoilage is an excess-cover approximation.** True FIFO unit-age tracking needs a per-series queue and cannot be vectorised at this scale.
- **The backtest rolls the evaluation origin, not retraining.** It measures how a fixed system behaves as the origin moves; it does not measure the value of retraining on a cadence.
- **Lower-tail calibration is poor**, as the coverage table shows. It matters little for decisions taken above the median, but it is real.

Correctly out of scope

Multi-tenancy, approval workflows, ERP connectors, an agentic layer and the remaining portals are absent by choice rather than by omission. They are features that public data cannot exercise and that would add surface without adding evidence for the argument the system exists to make.

Where to start reading the code

The argument	dqstack/decide.py - the critical fractile and the ledger
The contract	dqstack/models/dist.py - the predictive-distribution interface
The honesty	dqstack/backtest.py - paired inference across origins
The interface	ui/js/review.js and ui/js/plain.js
The design	ui/css/tokens.css
The record	CONTEXT.md - decisions, defects and traps



DQ-Stack. Forecasts that carry their own uncertainty, their own data quality, and their own price.